

# RELATORIO DO PROJETO

Tales of SG - RPG de terminal em Python

## 1. Identificacao do Projeto

|                     |                             |
|---------------------|-----------------------------|
| Nome do projeto     | Tales of SG                 |
| Tipo de sistema     | Jogo RPG de terminal        |
| Linguagem utilizada | Python                      |
| Integrantes         | Walber Lima, Joao e Kellvyn |

## 2. Objetivo do Sistema

O objetivo do projeto Tales of SG e desenvolver um jogo RPG simples em Python, aplicando conceitos estudados em sala, como variaveis, listas, dicionarios, funcoes, condicionais, repeticoes, tratamento de erros e organizacao de codigo.

No jogo, o jogador cria um personagem e atravessa bairros de Sao Goncalo em busca do lendario X-Tudo Supremo. Pelo caminho, enfrenta inimigos, derrota bosses, coleta itens, ganha experiencia e pode comprar equipamentos na feira de Alcantara.

A historia mistura aventura medieval com humor baseado em situacoes do cotidiano. A missao parece simples, mas atravessar Sao Goncalo por causa de um lanche nunca seria uma tarefa normal.

## 3. Funcionalidades Principais

- Criacao de personagem e escolha de classe.
- Tres classes jogaveis: Mago, Barbaro e Arqueiro.
- Sistema de batalha contra inimigos comuns.
- Sistema de batalha contra bosses por mapa.
- Progressao entre mapas apos derrotar o boss.
- Sistema de inventario e uso de pocoes.
- Loja localizada em Alcantara.
- Sistema de dinheiro, XP e subida de nivel.
- Drops aleatorios de itens apos batalhas.
- Tratamento de erros com try/except.
- Sistema anti-trapaca com o Cobrador de Impostos Statais.

## 4. Imports Utilizados

**random:** utilizado para gerar aleatoriedade no jogo, como dano, esquiva, drops, dinheiro recebido, vida dos inimigos e ataques dos bosses. Sem esse modulo, todo combate ficaria previsivel demais, quase igual fila que nunca anda.

**time:** utilizado para criar pausas na narracao e no texto lento, deixando a apresentacao dos dialogos mais parecida com um RPG classico.

## 5. Estruturas de Dados

O projeto utiliza listas e dicionarios para organizar as informacoes do jogo. A lista **inventario** armazena os itens do jogador. A lista **mapas** guarda os mapas, inimigos, bosses e progresso de cada regioao. O dicionario **jogador** guarda os atributos principais, como nome, classe, vida, ataque, nivel, XP e grana.

## 6. Funcoes Desenvolvidas

| Funcao               | Descricao                                            |
|----------------------|------------------------------------------------------|
| texto_lento()        | Exibe o texto letra por letra.                       |
| esperar()            | Pausa a narracao ate o jogador apertar ENTER.        |
| narrar()             | Combina texto lento com pausa de dialogo.            |
| introducao()         | Mostra a abertura e o inicio da historia.            |
| historia_mapa()      | Exibe a narracao de cada mapa.                       |
| criar_personagem()   | Permite criar o personagem e escolher a classe.      |
| mostrar_status()     | Mostra vida, ataque, nivel, XP e dinheiro.           |
| mostrar_mapa()       | Mostra os mapas e destaca o mapa atual.              |
| subir_nivel()        | Controla o sistema de evolucao por XP.               |
| usar_pocao()         | Permite recuperar vida com a Pocao SaBOUR Guarana.   |
| ataque_do_inimigo()  | Calcula ataques e chance de esquiva.                 |
| batalha()            | Controla o combate contra inimigos e bosses.         |
| avancar_mapa()       | Libera o proximo mapa apos derrotar o boss.          |
| dropar_item()        | Gera dinheiro e possiveis itens apos batalhas.       |
| verificar_trapaca()  | Confisca excessos caso o jogador fique forte demais. |
| mostrar_inventario() | Exibe e permite usar itens do inventario.            |
| loja()               | Permite comprar itens em Alcantara.                  |
| menu()               | Controla a navegacao principal do jogo.              |

## 7. Sistema de Batalha e Balanceamento

As batalhas usam dano aleatorio para o jogador e para os inimigos. Cada classe possui atributos diferentes: o Mago tem mais ataque, o Barbaro tem mais vida e o Arqueiro possui maior chance de esquiva.

O sistema de drops foi balanceado para nao entregar itens o tempo todo. Assim, o jogador evolui durante a jornada, mas sem virar uma entidade divina antes de chegar no Centro.

Tambem foi criado o Cobrador de Impostos Statais, um sistema anti-trapaca que verifica se o jogador passou dos limites esperados de ataque, vida, dinheiro ou quantidade de itens. Caso isso aconteca, o excesso e confiscado e o jogador volta para um nivel considerado justo.

## 8. Tratamento de Erros

O jogo utiliza try/except para impedir que o programa quebre quando o usuario digita texto em menus que esperam numeros. Isso melhora a estabilidade do sistema e evita que o jogo seja derrotado por alguem digitando "abubeeble" no lugar de uma opcao valida.

## 9. Dificuldades Encontradas

- Organizar as funcoes sem baguncar o fluxo do jogo.
- Controlar a progressao dos mapas e impedir o jogador de pular bosses.
- Balancear vida, dano, XP, grana e drops.
- Criar uma narrativa com humor sem perder a logica da jornada.
- Implementar tratamento de erros sem deixar o codigo confuso.
- Testar o sistema de batalha, inventario, loja e anti-trapaca.

## 10. Conclusao

O projeto Tales of SG permitiu aplicar varios conceitos de Python em um sistema pratico e divertido. O jogo possui criacao de personagem, batalhas, mapas, inventario, loja, sistema de evolucao, drops, tratamento de erros e controle de balanceamento.

Mesmo sendo um RPG simples de terminal, o projeto demonstra organizacao de codigo, uso de funcoes, estruturas de dados e logica de programacao. No final, o jogador pode concluir sua jornada e finalmente encontrar o X-Tudo Supremo, que provavelmente era melhor na lenda do que na vida real.